

Redis架构

一、架构篇

面对高并发场景，提高查询返回速度

纯内存操作：Redis 启动后，直接向操作系统申请一块巨大的内存空间。它的所有数据读取、写入、更新，全部在这块内存里完成，绝不主动碰磁盘，至于持久化存储，是后台线程做的事情

使用Hash的方式：采用了最简单的 Key-Value（键值对）字典模型，复杂度永远是 $O(1)$

采用类似于Nginx的网络IO模型

可以把 `epoll` 想象成一个极其高效的“智能快递柜”。

10,000 个客户端连接，相当于 10,000 个快递柜格子。

Redis 根本不派 10,000 个保安去盯着格子。它只有一个主线程（处理网络请求和执行命令）。

`epoll` 机制会在底层帮 Redis 盯着这 10,000 个格子。只有当某个格子里真的有请求发过来了（网络可读/可写），`epoll` 才会立刻通知 Redis 的主线程：“第 88 号和第 1024 号格子来活了，去处理一下！”

Redis 主线程立刻飞奔过去，从内存里极速读写完数据，返回结果，然后再回到大厅等 `epoll` 的下一次呼叫。

但是也跟Nginx是有区别的，redis只是单线程，Redis跟Mysql最大的一个区别，就是没有锁机制

Redis 为什么不敢像 Nginx 那样开 8 个 Worker 一起干活？因为 Redis 是数据库！它的数据是**共享状态**的。如果 8 个线程同时去抢着修改同一个 Key 的值，立刻就会引发极其惨烈的数据错乱。为了解决错乱，就必须加锁；一旦加锁，性能就会暴跌，甚至死锁（这就走回了 MySQL 的老路）

能够卡住Nginx的是网络带宽，Redis只会因为处理复杂度较高导致主线程单个卡死

二、数据结构篇

总的来说，不管说使用了什么数据类型，都是为了方便排序，方便查找，没有像B+树的那样花里胡哨，一切都是简单、直接、暴力，当然，为了解决链表的弊端，采用多层跳表的形式，方便更快地查找

Redis 发明了 Hash 结构，网络开销几乎为零，时间复杂度极速的 $O(1)$

list列表：结合了双向链表，并且保留了顺序，在两段插入和弹出数据的的速度永远是 $O(1)$ ，总的来说就是减少排序开销，而且该结构可以实现天然有序。相对于Mysql来说，它存的数据，是存在一颗B树的叶子节点，按照时间顺序取出数据时，都会进行复杂的计算，Redis就不需要，直接取出，天然有序

set集合：底层就是Hash，自带去重，用于求交集这种复杂运算

ZSet有序集合：将数据排序好，底层排序是用的跳表（适合范围查询）

三、并发与线程

痛点：

假设现在双十一大促，Redis 单线程（主厨）正在以每秒 10 万次的极速处理着用户的抢购请求。

突然，一个毫无经验的新手程序员，为了排查问题，在生产环境的客户端里敲下了一行极其致命的指令：

`KEYS *` (翻译: 查出 Redis 里所有的 Key)

物理规律的惩罚:

要知道, 当时 Redis 里可能有 5000 万个 Key, 这个指令的时间复杂度是 $O(N)$ 。

单线程接到这个命令后, 没有任何办法, 只能去仓库里挨个遍历这 5000 万个 Key。这可能要耗费他整整 10 秒钟

灾难降临: 因为是单线程, 这 10 秒钟内, 主厨被彻底卡死 (Block)。后面排队的 100 万个双十一抢购请求全部超时, 前端页面全部报错, 流量瞬间砸穿防弹衣, 直击后端的 MySQL。MySQL 当场宕机, 公司直接损失惨重。

SCAN与惰性删除: 通常会禁用KEYS命令, 转而使用SCAN命令, 每次只遍历某一小部分, 然后停下来去处理别人的抢购请求, 再回来接着遍历下一小部分; 如果要删除庞大的Hash表, 用传统的DLE也会卡死, redis引入unlike, 主线程负责将目标从目录上划掉, 然后交给后台的一个异步线程去慢慢清理这 100万个元素

redis并非完全抵制多线程, 而是在一个主线程 (该线程负责主要的核心处理部分) 的基础上, 对于网络的读写和字节的打包, 都交给其他线程处理

四、持久化

Redis 在内存里狂飙到 10 万 QPS, 爽是爽了。但只要机房电闸一拉, 内存里 5000 万个 Key 灰飞烟灭。等来电重启后, Redis 空空如也, 所有请求全部砸向背后的 MySQL, 导致系统级雪崩。

解法: 内存快照

Redis 会每隔一段时间 (比如 5 分钟), 利用操作系统的 `fork()` 机制克隆出一个后台子进程。这个子进程会把当前内存里所有的极其复杂的数据结构 (不管你是跳表还是哈希表), 全部压缩成一个极其紧凑的二进制文件——`dump.rdb`, 然后悄悄扔到硬盘上。

优势: 这是“物理级别的备份”。万一宕机, 重启时 Redis 只要把这个 `.rdb` 文件重新读进内存, 瞬间满血复活! 恢复速度极其变态, 这也是很多公司用它做冷备的原因。

但是, 光有快照还不够, 因为间隔是5分钟, 其中也会丢失很多数据, 于是有了AOF追加日志, 就算下一秒断电, 重启时主厨只要翻开这个 AOF 文本, 把里面的命令像复读机一样从头到尾重新执行一遍, 数据就一根毛不差地回来了

同时, AOF记录的数据到达一定数量, 会创建新的AOF文件, 防止AOF文件占用过大的内存

五、内存淘汰和过期策略

在 Redis 里, 很多 Key 是带了“保质期 (TTL)”的 (比如验证码 5 分钟后失效, 或者分布式锁 10 秒后释放)

主线程平时不管保质期, 只有当客户端发来请求, 比如 `GET code:123` 时, 主厨才会顺手看一眼它的保质期。如果发现过期了, 当场斩杀, 返回空值。这叫“不碰就不杀”, 极大地节省了 CPU 对于主线程永远不会用到的key, 会有专门的后台进程来进行随机抽查并进行清理, 如下

内存淘汰: “过期策略”只是清理垃圾。但如果大家塞进来的全是不带保质期的永久数据, 或者突发流量太大, 把 8GB 内存彻底打满了, 连 1KB 都塞不进去了, 怎么办?

如果是mysql或者是操作系统里面，就会有冷热数据分离，或者LRU等等，去清理内存页内存，但是mysql里面是管理的数据页，页数并不多，而redis里面的key动辄5000万，就算一个一个淘汰，还是很慢

redis的LRU策略

Redis 会在每个 Key 的对象头里，偷偷借用 24 个 bit（极小的空间），记录下这个 Key 最后一次被访问的时间戳。

当内存满了需要杀人腾位置时：

Redis 不去遍历所有数据找最老的。

而是随机抽取 5 个 Key（数量可配置）。

看看这 5 个 Key 里谁的时间戳最老，直接一刀砍死！

这种“随机抽样+局部淘汰”的办法，用极小的内存开销，达到了和经典 LRU 几乎 80% 相似的效果。

另外，冷热数据分离，可能导致缓存污染（mysql里面）

而redis是使用的被访问的频率，来判断数据的“冷热”

六、三大灾难

缓存穿透 Cache Penetration

- **痛点（黑客的狂欢）：**

假设你的系统 URL 是 `get_user?id=1`。正常的流程是：先去 Redis 查 `id=1`，查不到，再去 MySQL 查，查到了放回 Redis。

突然，黑客发起了僵尸网络攻击，疯狂请求 `id = -1` 或者极其离谱的不存在的数据。

灾难发生：Redis 里肯定没有 `-1`，于是请求全部穿透 Redis 砸向 MySQL。MySQL 去 B+ 树里找，也找不到，于是也没办法把结果写回 Redis。

结果就是：这件防弹衣彻底失效了！黑客的千万级并发像利剑一样，次次都精准刺穿 Redis，直接把后端的 MySQL 捅死。

- **解法 1：缓存空对象（打个绷带）**

MySQL 查不到是吧？我也硬塞一个 `key: -1, value: null` 到 Redis 里，设置个 30 秒过期。下次黑客再查 `-1`，Redis 直接返回 null，挡住了！

新痛点：如果黑客每次用的 ID 都不一样（UUID），Redis 会被塞满无数个毫无意义的 null 垃圾。

- **终极绝杀：布隆过滤器（Bloom Filter）**

在请求到达 Redis 之前，加一道极其神奇的“安检门”——布隆过滤器。

它利用多个 Hash 函数和极其微小的位图（Bitmap），把全量真实存在的 ID 映射成一串 0 和 1。

它的神仙特性是：如果它说这个 ID “存在”，那这个 ID 大概率存在（有极小误判）；但如果它说这个 ID “不存在”，那这个 ID 绝对不存在！

黑客拿假 ID 一冲，布隆过滤器瞬间判断“绝对不存在”，直接在最外层把请求丢弃，连 Redis 都不用进，完美防御！

- **痛点（单点破防的悲剧）：**

双十一零点，有一款“iPhone 15 Pro Max 1元秒杀”的商品，它是全网最顶级的**超级热点 Key**。

无数的请求（每秒 10 万并发）全部盯着这个 Key 疯狂读取。

突然，**这个 Key 的保质期（TTL）到了，恰好在零点零一秒过期了！**

灾难发生：就在这一瞬间，Redis 里的防弹衣破了一个极小的“洞”。但这 10 万个并发请求瞬间透过这个洞，同时发现“缓存没了”，于是它们**同时**转头去向 MySQL 发起查询，试图把数据重新加载到缓存。

MySQL 根本扛不住这瞬间的 10 万个相同查询，当场 CPU 100% 宕机。这就叫击穿（一个点被瞬间打穿）。

- **解法：互斥锁（Mutex Lock / 分布式锁）**

当这 10 万个请求发现缓存没了时，**绝对不允许它们一窝蜂去查数据库。**

让它们先去抢一把锁（比如 Redis 的 `SETNX`）。

- 抢到锁的那个幸运儿（1个请求），才有资格去 MySQL 查数据，并把数据重新写回 Redis。
- 剩下没抢到锁的 99,999 个请求，乖乖原地睡眠（Sleep）几十毫秒，醒来后重新去查 Redis。此时数据已经写回来了，大家皆大欢喜。

- **痛点（大面积崩盘）：**

“击穿”只是破了一个洞，而“雪崩”是防弹衣彻底碎了。

发生的原因通常有两个：

1. **Redis 物理宕机了**（主厨直接罢工）。

2. 程序员在写代码时，为了省事，给几百万个商品设置了**完全相同的过期时间**（比如都是12小时后过期）。结果时间一到，几百万个 Key **在同一秒钟集体过期**！

灾难发生：防弹衣瞬间消失，铺天盖地的各类请求像雪崩一样，没有任何阻挡，全部砸向底层极其脆弱的 MySQL。

- **解法 1：打散过期时间（防聚集）**

极其简单却极其有效的一招：在设置 TTL 时，给基础时间加上一个**随机数**（比如 12小时 + 随机 1~600 秒）。让这批 Key 均匀地在 10 分钟内陆续过期，把泄洪的压力彻底平摊，MySQL 完全扛得住。

- **解法 2：Redis 高可用集群（防宕机）**

如果是因为 Redis 宕机引发的，那就必须上**架构级防御**：引入主从复制（Master-Slave）、哨兵集群（Sentinel）或者切片集群（Redis Cluster）。主节点挂了，几秒钟内自动提拔从节点顶上，保证防弹衣永远在身上。

七、面试题

为什么 Redis 那么快？

1. **完全基于内存：** Redis 的绝大部分请求是纯粹的内存操作，极其快速。由于数据存储在内存中，这使得其查找和操作的时间复杂度通常可以达到 $O(1)$ ，直接绕开了磁盘 I/O 的物理速度瓶颈。
2. **高效的数据结构：** Redis 并没有直接使用 C 语言传统的字符串或数组，而是为了性能专门设计了多种高效的数据结构（如简单动态字符串 SDS、字典 Hash、跳跃表 SkipList、压缩列表 Ziplist 等）。这些数据结构在时间效率和空间占用上都做了极致的优化。
3. **单线程避免了多线程开销：** Redis 的核心键值对读写是单线程的（注：Redis 6.0 虽然引入了多线程，但也仅用于处理网络 I/O 的数据读写和解析，核心命令的执行依然是单线程）。单线程设计不仅避免了线程创建、销毁和上下文切换带来的庞大开销，还彻底杜绝了多线程环境下的锁竞争问题，不存在加锁、释放锁以及死锁造成的性能损耗。
4. **优秀的 I/O 多路复用机制：** Redis 基于 Reactor 模式开发了高效的网络事件处理器。它通过 I/O 多路复用技术（如 Linux 下的 `epoll`）来同时监听多个 Socket 连接，并在单线程中非阻塞地处理这些请求，这使得 Redis 能够从容且高效地应对海量的并发网络连接。

Redis 是基于内存的，如果断电宕机数据怎么保证不丢？

RDB内存快照：

在指定的时间间隔内，Redis 会把当前内存中的所有数据集生成一个紧凑的二进制文件（通常叫 `dump.rdb`），保存到磁盘上，重启时直接把文件按原样加载进内存即可，速度远快于 AOF 但是容易丢失时间间隔内的数据

AOF追加日志：

Redis 会把每一个写操作的命令（比如 `SET`，`HSET`）以独立日志的形式追加记录到一个文件中（`appendonly.aof`）。当 Redis 重启时，会把这些命令从头到尾再执行一遍来恢复数据。但是恢复慢，文件体积容易膨胀 而且在恢复过程中，不再处理业务

如果 Redis 的内存写满了会怎样？说说 Redis 的内存淘汰机制/策略有哪些？

1. 内存满了之后，任何会导致内存增加的写操作（如 `SET`，`HSET` 等）都会直接报错（`OOM command not allowed`）。但是读操作（如 `GET`）可以正常进行
2. 从设置了过期时间的键中，挑选出最近最少使用的键淘汰、挑选出最不经常使用的键淘汰、从设置了过期时间的键中，挑选出存活时间最短（马上就要过期了）的键淘汰
3. 从所有键中，挑出访问频率少的。或者随机抽选，杀点相对访问频率较少的

2. 八大淘汰策略：其实只是一个“2 × 4 的矩阵”

很多候选人死记硬背 Redis 的 8 种淘汰策略，面试时背得结结巴巴。其实你只需要在脑海里画一个 2 × 4 的矩阵，瞬间就能全部记住并理解：

首先，确定“杀戮范围（作用范围）”（选 2）：

- 1 `volatile-*`（只杀带期限的）：仅在“设置了过期时间”的 Key 集合中进行淘汰。
- 2 `allkeys-*`（无差别屠杀）：在所有的 Key（不管有没有设置过期时间）中进行淘汰。

然后，确定“处决算法”（选 4）：

- 1 `random`（随机）：闭着眼睛瞎杀。极度不推荐，纯属碰运气。
- 2 `ttl`（快到期的先死）：专门针对 `volatile` 范围，谁的剩余寿命越短，就优先杀谁。
- 3 `lru`（Least Recently Used，最近最少使用）：看的是时间。谁最久没有被访问过，谁就滚蛋。
- 4 `lfu`（Least Frequently Used，最不经常使用）：看的是频率。谁被访问的次数最少，谁就滚蛋。（Redis 4.0 引入）

什么是缓存穿透、缓存击穿、缓存雪崩？分别怎么解决？

缓存穿透：客户端请求的数据，在 Redis 缓存里没有，在背后的 MySQL 数据库里也不存在，如果黑客利用这个漏洞，疯狂请求一个不存在的 ID（比如 `id=-1`），就会导致所有请求全部“穿透”缓存，直接把数据库打挂

解决：布隆过滤器，进行判断。或者标记空对象，设置一段时间内无法再次访问

缓存击穿：单个热点 Key 过期失效，导致出现高并发打向数据库

解决：加锁，只允许拿到锁的单个线程去访问数据库，并写回 Redis，其他线程休眠，直接读新数据

缓存雪崩：这不是单个 Key 的问题，而是大面积的 Key 在同一时刻集中失效，或者更严重的情况——Redis 整个集群直接宕机了给缓存设置过期时间时，千万不要全部设成一样（比如全都 12 点过期）。必须在基础时间上加一个随机值（比如 1~5 分钟的随机波动），让它们均匀地在不同时间失效

解决：给缓存设置过期时间时，千万不要全部设成一样（比如全都 12 点过期）。必须在基础时间上加一个随机值（比如 1~5 分钟的随机波动），让它们均匀地在不同时间失效

如果是 redis 宕机，就要设计 redis 集群来实现高可用

还有哪些高级的数据结构？

Bitmap：极限压缩内存的布尔统计，其适合只有两个状态的巨量数据统计，非常节省内存

HyperLogLog（基数统计），用于统计一个集合中不重复元素的个数，比 set 更节约内存空间

Redis 和 MySQL 的数据怎么保证双写一致性？

先查 Redis，命中直接返回；没命中，去查 MySQL，然后把结果写入 Redis，再返回。

写请求（核心坑点）：先更新 MySQL 数据库，然后再删除 Redis 缓存。

为什么？

容易读到脏数据、旧数据，redis 终究是分离出来的缓存模块，原始数据发生变动，缓存自当进行更新，但双方之间的改动又会有误差延迟，比如线程 A 和 B 先后写数据库，但由于网络延迟，B 先更新了缓存，A 后更新了缓存，导致缓存里存了旧数据

写数据时，为什么是“删除”缓存，而不是“更新”缓存？

答：有两个原因。第一是懒加载思想，有些缓存数据不是直接查出来的，而是经过多张表连表计算出来的，如果每次修改 DB 都去算一次扔进缓存，但这个缓存一天都没人读，这就是极大的算力浪费；第二是并发写冲突，如果线程 A 和 B 同时更新数据，DB 的顺序是 A->B，但由于网络延迟，更新 Redis 的顺序变成了 B->A，缓存里就留下了脏数据。所以，不更新，直接删，让下一个读请求自己去算，是最稳妥的。

为什么必须是“先更新 DB，后删缓存”？反过来“先删缓存，后更新 DB”不行吗？

答：始终要以数据库DB为结果导向，先删缓存，可能在更新DB之前，又进行缓存，导致读了更新之前的脏数据